Sistema de Inventario

Aplicación Web Flask - Gestión de Repuestos y Accesorios para Teléfonos

Documento generado: 31/03/2026

Tabla de Contenidos

1. 1. Características del Sistema
2. 2. Arquitectura del Proyecto
3. 3. Estructura de Directorios
4. 4. Modelos de Base de Datos
5. 5. Controladores
6. 6. Servicios
7. 7. Vistas y Rutas
8. 8. Formularios
9. 9. Plantillas HTML
10. 10. Configuración
11. 11. Tareas Programadas
12. 12. Seguridad
13. 13. Pruebas Unitarias
14. 14. Migraciones de Base de Datos
15. 15. Instalación y Ejecución

1. Características del Sistema

- CRUD completo de productos con categorías, marcas, proveedores y precios en Bs/USD
- Gestión de modelos de teléfono con marcas predefinidas
- Compatibilidad muchos-a-muchos entre productos (pantallas) y modelos de teléfono
- Filtros avanzados por nombre, categoría, marca, precio, proveedor y nivel de stock
- Alertas automáticas de stock bajo por correo electrónico
- Informes periódicos del estado del inventario (mañana y tarde)
- Respaldo automático de la base de datos comprimido en ZIP
- Actualización automática de precios en Bs al cambiar la tasa de cambio
- Exportación a Excel del inventario completo
- Actualización rápida de stock desde la vista de lista
- Autenticación con Flask-Login y registro de auditoría de seguridad
- Cabeceras de seguridad HTTP con Flask-Talisman (CSP)

2. Arquitectura del Proyecto

La aplicación sigue el patrón **MVC (Modelo-Vista-Controlador)** con una capa de servicios adicional:

Cliente (Navegador): Interfaz web HTML/CSS/JS

Vistas/Rutas (Views): Controladores de rutas Flask (Blueprints)

Controladores (Controllers): Lógica de negocio (ProductoController, AlertaController)

Servicios (Services): Servicios externos (email, auditoría, backup)

Modelos (Models): Capa de datos SQLAlchemy ORM

SQLite DB: Base de datos relacional

3. Estructura de Directorios

```
inventario_app/
├── app/
│   ├── __init__.py          # Factory de aplicación
│   ├── models.py            # Modelos SQLAlchemy
│   ├── forms.py             # Formularios WTForms
│   ├── controllers/
│   │   ├── producto.py      # Lógica de negocio de productos
│   │   └── alertas.py       # Lógica de negocio de alertas
```

```
│   ├── services/
│   │   ├── alertas.py        # Envío de emails, informes, backup
│   │   └── audit.py          # Registro de auditoría
│   ├── views/
│   │   ├── auth_routes.py    # Login / Logout
│   │   ├── producto_routes.py # CRUD productos
│   │   ├── modelo_routes.py  # CRUD modelos
│   │   ├── config_routes.py  # Panel de configuración
│   │   └── alertas_routes.py # Vista de alertas
│   ├── templates/            # Plantillas HTML
│   └── static/               # CSS, JS, imágenes
├── migrations/               # Migraciones Flask-Migrate
├── tests/                    # Pruebas unitarias (180 tests)
├── logs/                     # Logs de auditoría
├── instance/                 # Base de datos SQLite
├── config.py                 # Configuración centralizada
├── run.py                    # Punto de entrada
├── requirements.txt          # Dependencias
└── .env                      # Variables de entorno
```

4. Modelos de Base de Datos

Producto

Campo	Tipo	Descripción
id	Integer	Clave primaria
nombre	String(100)	Nombre del producto
descripcion	String(200)	Descripción opcional
categoria	String(50)	pantalla, bateria, funda, cable, cargador, otro
marca	String(50)	Marca del producto
cantidad_stock	Integer	Stock actual (default: 0)
stock_minimo	Integer	Stock mínimo de alerta (default: 5)
proveedor	String(100)	Proveedor del producto
precio_mayor_bs/usd	Numeric(10,2)	Precio por mayor
precio_detal_bs/usd	Numeric(10,2)	Precio al detal
precio_tecnico_bs/usd	Numeric(10,2)	Precio técnico
modelos_compatibles	Relación M:N	Modelos compatibles

ModeloTelefono

Campo	Tipo	Descripción
id	Integer	Clave primaria
nombre	String(100)	Nombre único del modelo
marca	String(50)	Marca del teléfono

Compatibilidad

Campo	Tipo	Descripción
producto_id	Integer (FK)	Referencia a productos.id
modelo_id	Integer (FK)	Referencia a modelos_telefono.id

ConfiguracionSistema

Campo	Tipo	Descripción
id	Integer	Clave primaria
clave	String(50)	Clave única de configuración
valor	String(255)	Valor de configuración
updated_at	DateTime	Fecha de última actualización

Usuario

Campo	Tipo	Descripción
id	Integer	Clave primaria
username	String(64)	Nombre de usuario único
password_hash	String(256)	Hash de contraseña (werkzeug)

rol	String(20)	Rol del usuario (default: admin)
is_active	Boolean	Estado de la cuenta
created_at	DateTime	Fecha de creación

5. Controladores

ProductoController (app/controllers/producto.py)

obtener_todos(): Lista todos los productos ordenados por nombre

obtener_con_filtros(): Filtra por nombre, categoría, marca, precio, proveedor, stock

obtener_por_id(id): Obtiene un producto con sus modelos compatibles

crear(data, modelos_ids): Crea un producto y notifica si el stock es bajo

actualizar(id, data, modelos_ids): Actualiza un producto existente

eliminar(id): Elimina un producto

actualizar_stock_rapido(id, nueva_cantidad): Actualización rápida de stock

AlertaController (app/controllers/alertas.py)

obtener_productos_bajos(): Productos con stock <= mínimo

obtener_productos_bajos_count(): Conteo de productos con stock bajo

obtener_ultima_alerta_global(): Fecha de la última alerta enviada

guardar_fecha_global(): Registra la fecha de alerta actual

guardar_fechas_productos(productos): Registra notificación por producto

6. Servicios

Alertas (app/services/alertas.py)

verificar_stock_y_notificar(app): Verifica stock bajo y envía email

enviar_alerta_email(app, productos): Envía alerta de stock por correo

generar_informe_general(app): Genera informe completo del inventario

enviar_informe_general_email(app, productos, estadisticas): Envía informe por correo

realizar_backup_automatico(app): Comprime DB y envía por correo

Auditoría (app/services/audit.py)

configurar_audit_logger(app): Configura logger rotativo (5MB, 5 backups)

registrar_evento(accion, detalle, usuario): Registra eventos de seguridad

Eventos registrados: LOGIN_EXITOSO, LOGIN_FALLIDO, LOGOUT, ELIMINAR_PRODUCTO, MODIFICAR_CONFIGURACION

7. Vistas y Rutas

Blueprint	Prefijo	Rutas	Protección
auth_bp	-	/login, /logout	Pública
producto_bp	/productos	/, /crear, /editar/<id>, /eliminar/<id>, /actualizar-stock/<id>, /enviar-alerta-manual, /enviar-informe-manual, /exportar-excel	Login required
modelo_bp	/modelos	/, /crear, /editar/<id>, /eliminar/<id>	Login required
config_bp	-	/configuracion, /configuracion/jobs/actualizar, /configuracion/backup-manual	Login required
alertas_bp	-	/alertas, /alertas/enviar	Login required

8. Formularios

LoginForm

- username (requerido)
- password (requerido)
- remember_me (opcional)

ProductoForm

- nombre (requerido, max 100 chars)
- descripcion (opcional, max 200 chars)
- categoria (requerido: pantalla, bateria, funda, cable, cargador, otro)
- marca (opcional, 14 marcas predefinidas)
- cantidad_stock (requerido, >= 0)
- stock_minimo (requerido, >= 1)
- proveedor (opcional, max 100 chars)
- modelos_compatibles (checkboxes dinámicos desde BD)
- 6 campos de precio (mayor, detal, técnico en Bs y USD)

ModeloForm

- nombre (requerido, max 100 chars)
- marca (opcional, 14 marcas predefinidas)

9. Plantillas HTML

Plantillas del Sistema

Plantilla	Propósito
base.html	Layout base con navegación, sidebar, flash messages
auth/login.html	Formulario de inicio de sesión
listar.html	Lista de productos con filtros y acciones
producto/crear.html	Formulario de creación de producto
producto/editar.html	Formulario de edición de producto
modelo/listar.html	Lista de modelos con filtros
modelo/crear.html	Formulario de creación de modelo
modelo/editar.html	Formulario de edición de modelo
config/jobs.html	Panel de configuración de jobs, correo y tasa de cambio
alertas/index.html	Vista de productos con stock bajo
email/alerta_stock.html	Plantilla de email de alerta de stock
email/informe_general.html	Plantilla de email de informe general

10. Configuración

Archivo .env requerido en la raíz del proyecto:

```
SECRET_KEY=tu-clave-secreta-muy-larga-y-segura-aqui
DATABASE_URL=sqlite:///inventario.db
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=true
MAIL_USERNAME=tu-email@gmail.com
MAIL_PASSWORD=tu-contraseña-de-aplicacion
MAIL_DEFAULT_SENDER=tu-email@gmail.com
FORCE_HTTPS=false
TASA_CAMBIO_USD_BS=0
FLASK_DEBUG=true
```

11. Tareas Programadas

El sistema utiliza **Flask-APScheduler** para ejecutar 4 tareas automáticas configurables desde el panel de configuración:

Jobs Programados

Job	Función	Default	Descripción
Alerta Stock Bajo	verificar_stock_y_notificar	8:00 AM	Envía email si hay productos con stock <= mínimo
Informe Mañana	generar_informe_general	7:00 AM	Envía informe completo del inventario
Informe Tarde	generar_informe_general	7:00 PM	Envía informe completo del inventario
Backup Diario	realizar_backup_automatico	9:00 PM	Comprime la DB y la envía por email

12. Seguridad

Características de Seguridad

Característica	Implementación
Autenticación	Flask-Login con sesiones seguras
Contraseñas	Hash con werkzeug (scrypt)
CSRF	Protección automática con Flask-WTF
Cabeceras HTTP	Flask-Talisman con Content Security Policy
Auditoría	Logger rotativo en logs/security.log
Autorización	@login_required en todas las rutas protegidas
Open Redirect	Validación de parámetro next en login
Clave secreta	Validación de longitud mínima (32 chars)

13. Pruebas Unitarias

El proyecto cuenta con **180 pruebas unitarias** organizadas en 7 fases:

Resumen de Pruebas

Fase	Archivo(s)	Tests	Cobertura
1. Infraestructura	conftest.py, factories.py	7	Fixtures y helpers
2. Modelos	test_models/*.py	45	100% models.py
3. Formularios	test_forms/*.py	32	100% forms.py
4. Controladores	test_controllers/*.py	44	94% producto.py, 100% alertas.py
5. Servicios	test_services/*.py	12	98% alertas.py
6. Vistas/Rutas	test_views/*.py	40	Integration tests
Total		180	

Comandos de Prueba

pytest - Todas las pruebas

pytest tests/test_models/ - Una fase específica

pytest tests/test_controllers/test_producto_controller.py - Un archivo específico

pytest tests/test_controllers/test_producto_controller.py::TestCrear::test_crear_minimo - Una prueba individual

pytest --cov=app --cov-report=term-missing - Con cobertura

pytest -v - Verbose

14. Migraciones de Base de Datos

flask db migrate -m "descripción del cambio" - Crear nueva migración

flask db upgrade - Aplicar migraciones

flask db downgrade - Revertir última migración

15. Instalación y Ejecución

Requisitos

Python 3.10+

pip

Instalación

16. cd inventario_app
17. python -m venv venv
18. source venv/bin/activate # Linux/macOS
19. pip install -r requirements.txt

Ejecución

Desarrollo: python run.py

Producción: gunicorn --bind 0.0.0.0:5000 run:app

La aplicación estará disponible en **http://localhost:5000**